



International Islamic University Chittagong
Department of Computer Science and Engineering

A Thesis Proposal on
CacheFlow: Memory-Aware Expert
Scheduling for Efficient
Mixture-of-Experts Models

Submitted By:

Mir Md. Tarhimul Quader	C221017
Md. Ifthekar Ahamed Sayem	C221020
Turja Dutta	C221026

Approval of Supervisor:

Approval of Co-Supervisor:

Jamil As-Ad
Lecturer
Department of CSE, IIUC

Arif Mohammad Asfe
Adjunct Lecturer
Department of CSE, IIUC

Abstract

Large language models trained today commonly use Mixture-of-Experts (MoE) architectures, which divide token processing among a number of expert sub-networks, resulting in high performance, though at the expense of high GPU memory requirements. These restrictions prevent researchers and developers to utilize such models on consumer grade hardware. To alleviate this problem, this thesis proposes a concurrent expert-scheduling and caching scheme that dynamically rotates experts in GPU memory, defers tokens to prevent pipeline stalls, and uses a hybrid scheduler which prioritizes expert loading based on demand, confidence, and wait-time. Built in PyTorch, the system will be tested on different cache sizes and hardware systems, including NVMe-supported systems to pure CPU RAM-limited systems, and the result will be measured in latency, throughput, GPU usage, and cache hit rate. The aim is to make MoE inference efficient and accessible, which opens the way to scalable, modular AI inference on low-cost hardware via high-throughput, low-latency inference without necessarily having all experts permanently in GPU memory.

Contents

1	Introduction	3
2	Problem Statement	3
3	Objectives	4
4	Literature Review	4
5	Proposed Methodology	6
5.1	Model Architecture	6
5.1.1	Token Scheduling Engine	7
5.1.2	Expert Caching Strategy	7
5.1.3	Execution Loop and Queue Management	7
5.1.4	Evaluation and Benchmarking	7
5.2	Dataset and Workload Design	8
5.3	Comparative Evaluation	8
5.4	Implementation Details	8
5.5	Expected Contributions	9
6	Expected Outcomes	9
6.1	Efficient Concurrency	9
6.2	Scalable Expert Management	9
6.3	Dataset-Backed Evaluation	10
6.4	Practical Deployment Tool-chain	10
6.5	Research Contribution	10
7	Conclusion	10
8	Work Distribution	11
9	Timeline	11

1 Introduction

Over the past few years, large language models (LLMs) have grown at an unprecedented pace, powering applications from conversational AI to code generation. This progress, however, has come at a steep cost: models now consist of billions of parameters, requiring immense computational resources to train and deploy. As researchers sought to make these systems more efficient, Mixture-of-Experts (MoE) architectures emerged as a breakthrough innovation. By activating only a small subset of specialized “experts” for each token, MoE models achieve massive capacity while keeping inference costs relatively low.

Yet this remarkable efficiency introduces a new challenge. The very mechanism that makes MoE scalable—the selective routing of tokens to only a few experts at a time—creates a bottleneck in execution. In real deployments, this leads to underutilized hardware, token scheduling inefficiencies, and frequent stalls in the inference pipeline. Systems like vLLM have made important strides by optimizing the key-value (KV) cache to accelerate attention computation, but these optimizations only address part of the problem. The deeper issue of expert-level scheduling and resource management remains largely unsolved.

This thesis takes aim at that gap. We propose a concurrent token scheduling and dynamic expert caching framework designed to make MoE inference not only efficient but also scalable on modest hardware. By intelligently deciding which experts to keep loaded in GPU memory, replacing underutilized ones, and processing tokens concurrently, the system enables high throughput without requiring all experts to remain resident. The inference pipeline becomes fluid: tokens are routed, processed, and recombined seamlessly, reducing idle time and unlocking the full potential of MoE’s sparsity.

The payoff is significant. With this approach, even consumer-grade machines can host and serve models that were once reserved for specialized, high-end infrastructure. By addressing the hidden inefficiencies in MoE execution, this work aims to bring state-of-the-art language modeling closer to universal accessibility.

2 Problem Statement

Although MoE models decrease compute cost by only activating a small amount of experts per token, their entire parameter sets still have to be made accessible during inference, resulting in latency and GPU under-utilization when the experts are not stored on-GPU. Existing caching policies such as Least Recently Used (LRU) cannot accommodate highly dynamic expert access patterns, leading to inefficient scheduling and many stalls. These problems point to a basic gap

How do we construct a memory-sensitive expert-caching and token-scheduling framework to support high-throughput and low-latency inference in large models of experts under stringent VRAM constraints?

3 Objectives

- Concurrent scheduling enables efficient token execution by prioritizing tokens mapped to already cached experts, reducing idle GPU cycles.
- Dynamic expert caching maintains a rotating set of high-demand experts in GPU memory, ensuring optimal resource usage under VRAM constraints.
- Priority-based loading policies introduce a scoring mechanism that combines token demand, model confidence, and load cost to guide expert replacement.
- Comprehensive evaluation benchmarks the system across varying cache sizes, hardware configurations, and workloads to ensure robust performance validation.
- Practical deployment delivers a modular, PyTorch-based framework for scalable MoE inference in both research and industry settings.

4 Literature Review

Vaswani et al. [1] introduced a novel architecture replacing recurrence and convolution with self-attention mechanisms, enabling long-range dependency modeling and fully parallelized training. The design made it possible to learn long-range dependencies across sequences and permitted full parallelization of training, resulting in major advances in machine translation and other sequence-to-sequence tasks. The model minimized training time by getting rid of repeated computations and enhanced gradient flow. Nevertheless, the method required substantial computing, and sizeable datasets, making it inaccessible to smaller research groups. Nevertheless, the architecture provided a general prototype on which future studies in generative pre-training and large-scale language models would be based.

Radford et al. [2] demonstrated the potential of unsupervised generative pre-training to significantly improve performance on several NLP benchmarks through transfer learning. The approach resulted in significant gains in the benchmarks of question-answering, sentiment-analysis and textual-entailment by pre-training a model on large unlabeled text corpora and fine-tuning it to downstream tasks. This paper demonstrated the power of transfer learning in NLP and demonstrated that general-purpose pre-trained representations could perform well compared to task-specific models. However, the intermediate model size and the need to use supervision fine-tuning restricted flexibility and scalability, which inspired the search to consider much larger pre-trained models that can achieve zero-shot generalization.

The development of very large generative models by Radford et al. [3] showcased impressive zero-shot capabilities across multiple language tasks with minimal task-specific adaptation. This model demonstrated high zero-shot performance on various tasks, such as summarization, translation, and question answering, and only task-specific prompts, instead of fine-tuning are needed. The findings indicated that model scale on its own was sufficient to induce the generalization abilities in the absence of labeled data. Nonetheless, the massive computational demands created accessibility issues, and the model might produce false or malicious results in certain situations, and ethical and safety implications

in large-scale language modeling.

Researchers optimized deep Transformer training by introducing Root Mean Square Layer Normalization, improving training stability and robustness (Zhang et al. [4]). Researchers minimized training variance by adding Root Mean Square Layer Normalization, and accelerated convergence, particularly in very deep architectures. The approach made training more stable and model robust and was computationally efficient. Nevertheless, it had not entirely tackled scaling challenges associated with very large models and there was room to continue to make architectural and training improvements.

Shazeer et al. [5] enhanced feedforward layers with Gated Linear Unit variants, increasing expressiveness and gradient propagation in sequence modeling tasks. These mechanisms added gating capabilities enabling more expressive feature representations and gradient propagation allowing more efficient gradient propagation. The results of experiments demonstrated improved performance in a variety of NLP tasks, especially on the tasks of capturing the long-range dependencies and intricate interactions in sequences. The method, though, marginally amplified the computational complexity and overhead in parameters, and encouraged additional studies into the equilibrium between expressiveness and efficiency.

Dosovitskiy et al. [6] applied Transformer architectures to computer vision, treating images as sequences of patches and achieving competitive classification results. This method demonstrated image classification performance competitive with, and even better than convolutional neural networks on image classification benchmarks. The procedure has shown the flexibility of self-attention in the modeling of long-range dependencies in various modalities. Although it was effective, it demanded very large datasets and computational resources to produce best results, which restricted its real life applicability in resource-constrained environments.

Fedus et al. [7] proposed sparsely activated models, allowing only a subset of parameters to be engaged per input and effectively scaling models to trillion-parameter sizes. Per input, only a sub-set of parameters (known as experts) was activated, minimising the computation but not model capacity. This sparse activation allowed large models to reach state-of-the-art performance on natural language tasks with a proportional reduction in training cost. However, these models created problems with stability of training, managing memory, and efficiency of inference, which prompted further developments in expert routing and resource-conscious serving.

Touvron et al. [8] explored lightweight foundational models on curated datasets, achieving competitive performance with lower computational demands. It was shown using lightweight trained foundational models on curated datasets that carefully optimized architectures could perform competitively with much lower computational demands. These open models enabled more experimentalism and deployment but were limited in terms of extremely large-scale inference and complex real time requirements, making resource-sensitive strategies and sparse model management desirable.

Dynamic expert swapping was introduced by Lepikhin et al. [9] to allow large MoE models to operate efficiently on memory-constrained devices. These models flexible in

deployment as they offloaded experts between the memory of GPUs and the CPU depending upon workload, ensuring performance. Although it was effective, the constant shuffling between levels of the memory caused overhead in communication, so perhaps more efficient caching and scheduling mechanisms are needed.

Sparsity-conscious caching mechanisms proposed by Roller et al. [10] significantly reduced inference latency for large MoE models while managing expert usage. This method dramatically decreased latency in inference, and allowed very large MoE models to be executed with personal machines. Although it was improved, the imbalance in the use of experts and penalties for cache miss continued to limit efficiency, leading to the creation of more adaptive routing and memory management strategies.

Dao et al. [11] developed paged attention, improving memory handling and throughput for very long sequence modeling in Transformers. This minimized memory bottlenecks and better throughput of large language models, but the technique introduced extra memory management overhead and careful tuning was necessary to avoid latency spikes, underscoring the importance of streamlined deployment methods.

Prompt-aware expert offloading strategies by Zoph et al. [12] selectively activate necessary experts per task, enhancing inference speed and memory efficiency. This approach enhanced memory efficiency and speed of inference because it activated only required components with each prompt. Although the strategy improved computational performance and ensured high performance, routing overhead and load balancing in heterogeneous settings were still a challenge, stimulating the further development of adaptive MoE inference mechanisms.

Narang et al. [13] refined MoE architectures to achieve high accuracy with reduced computational and memory costs, optimizing large-scale deployment. These models showed that architectural and routing optimization could ensure high accuracy with a low cost of computation and memory. Nonetheless, there were difficulties in expert deployment and low-latency inference at a wide range of deployment conditions, and research potentials in scalable and efficient model serving.

5 Proposed Methodology

5.1 Model Architecture

Proposed system is based on sparse MoE transformer where only few expert sub-networks are activated per token. Experts are each separate and loaded independently. Through this modularity we can dynamically build and schedule expert-modules at runtime depending on conditions, upon which runtime-aware inference can be built. An expected diagram of the proposed model architecture is given in the Figure 1:

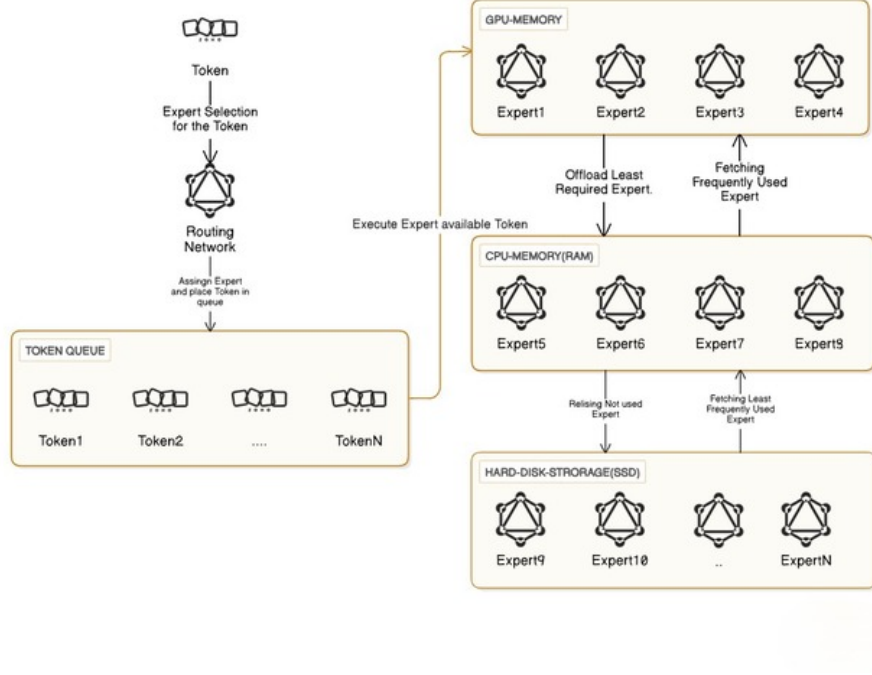


Figure 1: Proposed Model Architecture

5.1.1 Token Scheduling Engine

Central scheduler keeps tracking available tokens that can be processed with experts currently in cache. The tokens of experts not available are deflected to a deferral queue. This aspect makes concurrency a reality because it is possible to process the tokens independently, based on availability of expertise.

5.1.2 Expert Caching Strategy

The number of experts that can be loaded such that it does not use up the GPU VRAM is limited. We use caching where some expert set is stored in the memory. Experts are also added or removed to cache according to real-time usage and demand-wait time based priority score. It aids in the optimization of reuse and wholly useless loads of the disk.

5.1.3 Execution Loop and Queue Management

The workload of scheduling pipeline is orchestrated by the execution loop. It accesses both ready queues, deferred ones and asynchronously invokes background expert loads. Such configuration means the activity of GPUs can be kept online but other work done in parallel.

5.1.4 Evaluation and Benchmarking

At first, we will load our language model with various workloads to test its behavior by coming up with hand-crafted prompts and alternatively using prompts that will be applied in actual language model tutoring requests made by the students. Soon enough we will be comparing throughput, average token latency, expert reuse ratio to the old-school static scheduling strategies. Then we will perform a set of ablation experiments to determine

which components by themselves (scheduling techniques and caching techniques) are having the most effect.

5.2 Dataset and Workload Design

As a realistic point of reference, we intend to apply the same data set that the **vLLM PagedAttention** article used, which has become the standard amongst all to test LLM inference. It contains a collection of various sequence of texts that replicate real-world inference scenarios, allowing us to test how the system performs at different text sequence lengths, distributions of tokens and concurrent requests. With this time-tested benchmark, we can compare our performance directly with state of the art tools such as vLLM.

In addition to that dataset we will also prepare some artificial workloads to truly stress the expert swapping and cache mechanics, and ensure that the scheduler itself is solid when expert requests are frequent.

5.3 Comparative Evaluation

In contrast to the situation of the static inference systems where all experts are loaded, our system only maintains a small number of experts in the GPUs memory at any time. A token queue handles requests, and when a token requires an expert that is not loaded at the moment the scheduler will evict the least-used expert that is already loaded in the cache and load the request expert. The replacement strategy is dynamic and causes the GPU memory usage to be optimum and the inference pipeline does not stall.

In the evaluation we are going to compare our approach with three baselines:

- **Static Pinning:** A starting point with all experts pre-loaded, only appropriate to small models.
- **Synchronous On-Demand Loading:** The conventional stall-based systems wait until lost experts load before resuming inference.
- **Simple LRU Caching:** A simple eviction policy which does not take into account token demand or load costs.

We will compare our scheduling algorithm to demonstrate that concurrency and dynamic caching reduced latency and improved throughput.

5.4 Implementation Details

The system will be written in **PyTorch** and made as a modular system. The important sections here are as follows:

- **Dynamic Expert Loader:** Experts are stored out of memory and loaded into it on-demand to provide finer control over the use of a graphics card.
- **Intelligent Scheduling Algorithm:** A score-based algorithm determines what expert is evicted, based on demand of the token, frequency of use of an expert, and memory constraints.

- **Concurrent Processing:** We will overlap token processing with expert loading using CUDA streams and asynchronous I/O and reduce idle time on the GPU.
- **Scalable Design:** The code can be configured to scale with ease to a distributed environment and therefore you can experiment with multi-gpu or multi-node system.
- **Instrumentation and Profiling:** We will measure real-time metrics such as latency, cache hit rate, GPU utilization and expert eviction frequency.

5.5 Expected Contributions

This study will provide a couple of important items:

1. A dynamic scheduling and caching system of MoE inference that drives high throughput with limited VRAM on hardware.
2. A PyTorch-based, modular, implementation, which enables the concurrent loading and execution of experts.
3. A fair, standardized test on datasets used in the vLLM PagedAttention paper, to be able to compare it meaningfully to existing frameworks.
4. Insights into future work with large-scale inference that are based on expert caching strategies and eviction policies.

6 Expected Outcomes

By the time we close this we anticipate the project to have the following outcomes:

6.1 Efficient Concurrency

Handling tokens simultaneously, we ought to not only cut the average latency by half but also make the hardware perform better than the previous synchronous systems. Having fewer tokens held waiting on the cache and expert allocation flipping on the fly, the inference engine is able to achieve near-optimal throughput even with the limited memory.

6.2 Scalable Expert Management

Our cache-based scheme addresses the VRAM bottleneck of giant MoE models. Rather than loading all the experts into GPU memory, we maintain a small repertoire in memory and the others are swapped into place as scheduled using a token-based scheduling. This keeps the high-utilization experts alive as well as allows the low-utilization ones to die away without affecting performance. This means that models previously requiring more than 10 Gb of VRAM can be run on regular GPUs with only 1-2 Gb of additional overhead.

6.3 Dataset-Backed Evaluation

In order to ensure our work is practical, we will be using the same dataset in the vLLM PagedAttention [11] paper. That allows us to directly compare cache performance, schedule overhead, and the throughput to known implementations, providing a comparable and reproducible analysis.

6.4 Practical Deployment Tool-chain

We are assembling a PyTorch toolkit which is modular and contains reference code both to the caching mechanism and the scheduling algorithm. It will release scripts to train, infer and benchmark it so that researchers and practitioners can modify or adapt the framework to their own MoE pipelines.

6.5 Research Contribution

The system contributes to the existing debate as to how to scale modular neural networks efficiently. Combining cache-sensitive expert management with token-sensitive scheduling, it provides a viable architecture that reduces memory usage, increases parallelism and decreases the hardware cost of implementing MoE systems. The experimental findings will also cast a light on the trade-offs between memory footprint, scheduling complexity and inference speed, providing future large-scale optimization directions.

7 Conclusion

This research aims to bridge the gap between the theoretical scalability of Mixture-of-Experts (MoE) models and their practical deployment on resource-constrained hardware. While MoE architectures promise unprecedented model capacity without linearly increasing inference costs, their deployment remains heavily bottlenecked by the limitations of GPU memory and inefficient token-to-expert execution pipelines. Our work introduces a memory-aware caching layer that maintains a carefully curated subset of experts in GPU VRAM, complemented by a dynamic expert scheduling mechanism that intelligently evicts and loads experts based on demand, access frequency, and priority scoring. This is further enhanced by token concurrency optimizations, which ensure that token processing is never stalled unnecessarily, keeping GPU utilization high and inference latency low.

The proposed framework is designed not only to demonstrate proof-of-concept feasibility but also to provide a robust experimental foundation for future research on cache-efficient neural architectures. By systematically evaluating cache size, I/O bandwidth constraints, and token scheduling policies, this work offers deep insights into the trade-offs between model fidelity, throughput, and latency in sparse neural network inference. Beyond its immediate contributions, this research aims to create a modular, extensible toolkit that can be adopted by both academia and industry to deploy massive MoE models on commodity hardware. In doing so, it paves the way for cost-effective, democratized access to cutting-edge AI models, allowing smaller organizations, researchers, and developers to leverage large-scale models without prohibitive infrastructure investments.

Ultimately, this thesis positions itself at the intersection of systems optimization and deep learning architecture design, demonstrating that careful scheduling, caching, and concurrency engineering can unlock the full potential of sparsity in neural networks. By validating our approach with rigorous benchmarking and open-source tooling, we aim to make large-scale MoE models not only scalable in theory but truly deployable in practice.

8 Work Distribution

Team Member	Responsibilities
Mir Md. Tarhimul Quader	Evaluation pipeline; Benchmarking automation; Cache/I/O experiments; Evaluation section; Conclusion; Future work
Md. Iftaker Ahamed Sayem	Expert caching strategy; Scheduling algorithm; Async I/O pipeline; Literature review; Results; Discussion
Turja Dutta	Token scheduling engine; Execution loop; Profiling tools; Methodology writing; Abstract; Introduction

9 Timeline

Thesis Activity	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar
Literature Review												
Analysis of LLM Architectures												
Problem Framing												
Architecture Design												
Experiments and Evaluation												
Documentation												
Review and Revisions												
Final Submission												

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. Attention Is All You Need. *NeurIPS*, 2017. arXiv:1706.03762
- [2] Radford, A., Narasimhan, K., Salimans, T., Sutskever, I. Improving Language Understanding by Generative Pre-Training. *OpenAI Technical Report*, 2018. OpenAI
- [3] Radford, A., Wu, J., Child, R., et al. Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*, 2019. OpenAI
- [4] Zhang, B., Sennrich, R. Root Mean Square Layer Normalization. *NeurIPS*, 2019. arXiv:1910.07467
- [5] Shazeer, N. GLU Variants Improve Transformer. *arXiv preprint*, 2020. arXiv:2002.05202
- [6] Dosovitskiy, A., et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2020. arXiv:2010.11929
- [7] Fedus, W., Zoph, B., Shazeer, N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 2021. arXiv:2101.03961
- [8] Touvron, H., Lavril, T., Izacard, G., et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint*, 2023. arXiv:2302.13971
- [9] Zuo, S., Xue, F., Jin, H., et al. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. *arXiv preprint*, 2023. arXiv:2305.11794
- [10] Yoo, H., Lee, H. SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget. *arXiv preprint*, 2023. arXiv:2309.09515
- [11] Kwon, W., et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023. arXiv:2309.06180
- [12] Liang, W., et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv preprint*, 2024. arXiv:2405.04434
- [13] Shen, X., et al. fMoE: Fine-grained and Prompt-aware Expert Offloading for Large Mixture-of-Experts Serving. *arXiv preprint*, 2024. arXiv:2403.12881